Git Tutorial

Git is a free and open-source distributed version control system that tracks changes to files and allows for collaboration on projects. It's a fundamental tool for software development, enabling users to manage different versions of their code, track who made which changes and reverted to previous states if needed. Git works on your computer, but you also use it with online services like GitHub, GitLab, or Bitbucket to share your work with others.

Suppose a university software team is developing a Library Management System to manage books, track student borrowing, and calculate fines. The project has modules like Book Registration, Student Information, and Fine Calculation. Since new features are added frequently, proper version control is needed to keep track of changes and return to a stable version if required.

The team is using Git locally. They initialize the project with Git Bash, add and commit files with clear messages after every change. For example, if the Book Registration module is updated, they commit it with a note explaining the change. This helps maintain a clear project history and supports teamwork during development.

Phase 1: Getting Git on Your PC

Git Bash, which is the most common way to use Git on Windows.

Why do we need it?

- **Version Control:** You can manage different versions of your code and see who changed what.
- **Safety:** If you break a feature, you can "revert" to a previous stable state.
- **Cloud Sharing:** While Git lives on your computer, you use it to push your work to online services like **GitHub**, **GitLab**, or **Bitbucket**.

How to Install:

1. Go to git-scm.com.
2. Download the Windows installer.
3. Run the .exe file. Click "Next" through the defaults (the defaults are usually perfect for beginners).

4. Once finished, right-click any folder on your desktop. You should see **"Git Bash Here."**

Phase 3: The Big Concepts (Vocab)

Before we touch the commands, let's understand the "Library Management System" example. Imagine you have modules like "Book Registration" and "Fine Calculation". To keep them organized, you use these:

Core Concepts:

• Repository: A directory where Git tracks all project files and their history.
• Clone: Creating a local copy of a remote repository.
• Commit: Saving a snapshot of changes to the repository with a descriptive message.
• Branch: Creating separate lines of development to work on features or bug fixes independently.
• Merge: Combining changes from different branches into one.
• Push: Uploading local commits to a remote repository.
• Pull: Fetching changes from a remote repository and integrating them into the local repository.
• Staging Area: A temporary area where you stage changes before committing them.
• Remote Repository: A centralized repository hosted online (e.g., on GitHub, GitLab, Bitbucket).

- **Repository:** The main folder where Git tracks everything.
- **Staging Area:** A "loading zone." You put files here before officially saving them.
- **Commit:** A "snapshot" or a permanent save point with a descriptive note.
- **Push:** Sending your local "saves" up to the internet (GitHub).
- **Pull:** Grabbing the latest updates from the internet to your PC.

Basic Git Commands:

- git init: Initializes a new Git repository in the current directory.
- git push: Uploads local commits to a remote repository.

- git pull: Fetches changes from a remote repository and merges them.
- git commit -m "Your commit message": Saves staged changes to the repository.
- git branch: Lists, creates, or deletes branches.
- git clone : Clones a remote repository to your local machine.
- git status: Shows the current state of the working directory and staging area.
- git add : Stages a file for the next commit.
- git add . or git add -A: Stages all changes in the current directory.
- git checkout : Switches to a different branch.
- git merge : Merges changes from a specified branch into the current branch.
- git log: Displays the commit history.
- git diff: Shows the differences between files.

Command	Action	Location
git commit -m "..."	Take everything in the **Staging Area** and freeze it into a permanent snapshot/save point.	**Local Only.** This stays on your PC; no one else can see it yet.
git push	Take all your **Local Commits** and upload them to the internet (GitHub).	**Local ----> Remote.** Now your teammates can see your work.
git pull	Grab the latest work your teammates uploaded to GitHub and bring it onto your PC.	**Remote----> Local.** This keeps your PC up to date.

Phase 4: From Folder to GitHub (Step-by-Step)

Workflow Example:

1. Initialize: Start a new project or clone an existing one.

2. Add Files: Use git add to stage files you've modified or created.

3. Commit Changes: Use git commit to save the staged changes with a descriptive message.

4. Push Changes: Use git push to upload your local commits to a remote repository.

5. Collaborate: Use branches, pull requests, and merges to work with others. Adding a file to a repository

Follow this exact flow to put a project online:

1. Git bash

Go to the folder then Open the Git Bash

2.Configure User Info (only first time per computer)

git config --global user.name "Your Name"

git config --global user.email your_email@example.com

Check: git config --global --list.

4. Initialize Git

git init

3. Connect to GitHub (git remote add) ADD ORIGIN

You need to tell your local folder where its "home" is on the internet.

git remote add origin https://github.com/YourUsername/YourRepo.git

4. Stage the Files (git add)

To tell Git "I want to save these specific files," use:

git add . (The dot means "add everything in this folder")

5. Commit the file that you've staged in your local repository (git commit)

Now, lock those changes in with a message:

git commit -m "Any type of Message"

6. Push the files to the repository (git push)

Finally, send them to the cloud:

git push -u origin master

After a successful push, then go to your repository and click on refresh